

Received May 9, 2019, accepted June 2, 2019, date of publication June 12, 2019, date of current version June 27, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2922387

Prioritizing JUnit Test Cases Without Coverage Information: An Optimization Heuristics Based Approach

R. MUKHERJEE AND K. S. PATNAIK[✉]

Department of Computer Science and Engineering, Birla Institute of Technology, Mesra, Ranchi 835215, India

Corresponding author: K. S. Patnaik (kspatnaik@bitmesra.ac.in)

ABSTRACT Regression testing is an expensive activity and Test Case Prioritization (TCP) acts as an improvement mechanism for it. TCP techniques for object oriented programs need attention and in our study, we explored prioritization of JUnit test cases. Ten benchmark Java programs with their several mutated versions were studied. As collecting coverage information is a costly effort, we bypassed these steps and used optimization heuristics for ordering JUnit test cases at test method level. Our approach formulated a novel fitness objective which depends on the number of modified lines executed per unit of execution time. As regression testing is performed after some modification is done on an existing program, maximizing the execution of number of modified lines is highly lucrative. The test case prioritization problem was replicated in context of 0/1 Knapsack problem and then it was solved using Genetic Algorithm (GA). Our exploration also included application of Simulated Annealing and Ant Colony Optimization method for determining the best execution ordering of test cases. We examined the usage of Multi-objective GA by building another new fitness metric which aims to maximize the number of inheritance edges covered by a test case. Results indicate the superiority of optimization heuristics over other existing approaches. It appeared that multi-objective GA yielded better result than single objective prioritization. Among the single objective techniques, ACO performed best. To the best of our knowledge, this is the first study which explored all the above mentioned optimization heuristics for ordering JUnit test cases with the newly coined fitness intents.

INDEX TERMS Fault detected, fitness objective, optimization heuristics, regression testing, test method.

I. INTRODUCTION

Testing contributes minimum half of the effort while releasing a software and no matter how good the programming is, it still has one to three bugs per hundred statements [16]. Regression testing ensures no new errors have been introduced in a changed application. It confirms that the previous functionalities of a system are intact while the new changes are also working fine. Regression testing accounts for 80 % of the testing budget and 50% of software maintenance cost [15], [17]. Fixing a bug, meeting the demand of a changed requirement, improving functionality/features may act as the motivation behind changing a program. However, after every change, it is necessary to perform regression testing. But execution of all scheduled test cases is next to impossible because of time, cost, and resource constraints.

The associate editor coordinating the review of this manuscript and approving it for publication was Bijoy Chand Chatterjee.

While test case selection discards test cases, test suite reduction shows remarkable loss of fault detection rate [1]. Test Case Prioritization (TCP) overcomes these drawbacks and acts as an Improvement Testing mechanism. Test Case Prioritization method can be code element coverage based (statement/branch/function coverage etc.) or historical information based. Several Cost-Time centric TCP [11], Requirement-Risk cognizant TCP [13] and GUI/Web Application focused TCP [23] are also popular nowadays.

Prioritization techniques for object oriented programs are explored by very few studies [4], [24], [25]. In this work, we focused on analyzing TCP techniques for object oriented language Java by choosing 10 large open source Java programs (*Ant*, *JMeter*, *Xml-security*, *Jtopas*, *JDepend* etc.) as objects of study. The SourceForge (<http://sourceforge.net>) and Apache Jakarta (<http://jakarta.apache.org>) web sites helped us in collecting several versions of these programs. Eclipse plugin MuClipse was used to produce the

mutants of the programs under consideration. As JUnit (<http://www.junit.org>) is an extensively used framework for testing Java programs, we have decided to explore prioritization techniques for JUnit test suites. JUnit test suite has independently running test classes which in turn have many test methods. Identifying and reordering each test-method level test cases within a JUnit test class is more complicated than reordering test-class level test cases. JUnit test case sequence is basically a series of method calls. Literature survey shows very few experimentations [4], [26], [28], [29] on JUnit testing framework.

Review [21] of TCP techniques doesn't show any investigation on prioritization of JUnit test cases using optimization heuristics. We wanted to bridge this gap. As optimization algorithms produce near optimal or satisfactory approximate solutions in a reasonable time frame, many engineering design problems leverage them frequently. We applied several optimization heuristics (genetic algorithm, simulated annealing, ant colony optimization, MOGA) for solving the test case prioritization problem. In the very first step, the test case prioritization problem was replicated in context of 0/1 Knapsack Problem. Knapsack Problem is recognized as an NP-complete (hardest problem in NP) problem. The 0/1 knapsack problem states that given a knapsack with fixed capacity and a collection of discrete items each with its own weight and value, the maximum cumulative value of items that fills the knapsack should be such that the sum of the weight of the items does not cross the knapsack's capacity. We considered each test case as a knapsack item, the execution time of each test case as its weight, the number of modified lines executed by the test case as its value and the maximum amount of time required for executing the prioritized test suite as the knapsack's maximum capacity. After replicating the TCP problem in perspective of this problem, it was solved using *Genetic Algorithm*. Our study designed a novel fitness function which depends on the number of modified lines executed per unit of execution time. As regression testing is mostly implemented after some modification is done, maximizing the execution of number of modified lines is of utmost importance. Prioritization based on program change information was explored by Saha *et al.* [36] in 2015. Srivastava and Thiagarajan of Microsoft Research built a prioritization scheme Echelon which was based on changes made to a program [12]. The success of all these studies motivated us to design the new fitness metric.

Next, the probabilistic technique simulated annealing was applied for test case prioritization. To overcome the problem of converging on some local optima, we explored *simulated annealing* which also accepts worse solutions. Our study also investigated *Ant Colony Optimization* technique for TCP. While each ant moves from one node (test case) to another, the weight of each edge is determined by the amount of pheromone deposited with an evaporation rate of $k\%$ (used as 15% in this study). The realistic limitations of single objective regression test ordering has motivated us to explore *MOGA (Multi-objective GA)*. Apart from considering the previous

fitness metric, it included another objective of maximizing number of inheritance edges executed by a test case. As we are considering TCP for object oriented programs, taking object oriented features into account is highly important. As part of implementing MOGA, the widely recognized *NSGA-II* (Non-dominated Sorting Genetic Algorithm II) was run with a population size of 200 for 75 generations and the Pareto-optimal front was depicted.

We represented the outcome of each optimization heuristics with APFD (Average Percentage of Faults Detected) [1] metric and compared the results. It appeared that our study produced promising result in comparison with the results obtained by past studies [4], [26]. As gathering coverage information is a laborious activity, we hope the approach of ordering JUnit test cases using optimization techniques will become popular.

In a nutshell, this paper contributes the following research questions –

- **RQ1** - How several optimization heuristics can be applied for ordering JUnit test cases at test-method level?
- **RQ2** – Which optimization technique performs best for TCP?

The paper is structured as follows. Section II discusses basic facts and related work in Test Case Prioritization domain. Section III describes the research methodology with chosen Java programs, their size and origin. It also enlists the explored optimization techniques along with threats to validity. Section IV presents the strategy and operational details of our study. Section V analyses the key findings stating which optimization scheme works best. A number of statistical techniques (box plot, ANOVA etc.) along with graphical representation of APFD metric are narrated in this section. Section VI concludes our work with future scope.

II. TEST CASE PRIORITIZATION - BASICS AND RELATED WORK

In this section, we would like to report the origin of test case prioritization along with a remembrance of several important related works in this field.

Retest-All, Regression Test Selection (RTS), Test Suite Minimization (TSM) and Test Case Prioritization (TCP) are leading regression testing strategies [18]–[20]. The Retest-All strategy holds well when the test suite is small. However, as the size of test suite increases, an ordering mechanism becomes necessary. The goal of test case prioritization is multi-objective. Increasing fault detection rate or increasing capture of high priority requirements or decreasing the cost and time of prioritization mechanism are the primary aims of TCP.

Test Case Prioritization Problem Definition: For a test suite T discover T' in such a way that, $f(T') \geq f(T'')$, where f is the quantifiable performance or goal function, PT is the set derived from all possible prioritization orderings of T , $T'! = T''$, $T' \in PT$ and $T'' \in PT$.

The TCP problem has been addressed by several ways by many researchers [21]. Coverage aware prioritization techniques aim to maximize coverage of program elements (statement/branch/methods etc.) by a test case. They require detail knowledge of source code. Rothermel *et al.* [1], [5] showed that better coverage yields better fault detection rate. The shortcoming of their study was that it considered all faults of same severity and did not incorporate cost factor. In continuation with Rothermel's study, Elbaum *et al.* conducted another empirical assessment [3] which classifies the prioritization techniques as coarse granularity (function level techniques) and fine granularity (statement level techniques). In 2003, James A. Jones and M.J. Harrold for the first time applied Test Case Prioritization techniques on MC/DC test suite [2]. Studies also indicated that optimal technique is not that beneficial in terms of fault detection rate or execution time [7]. As regression testing is not a one-time activity, prioritizing test cases using historical information about test case performance record or historical fault information was also explored by researchers [8]–[10]. Cost centric prioritization schemes [11] focuses on building cost models by subdividing cost into several parameters like cost of analysis, cost of maintenance, cost of execution etc. Time aware TCP techniques [22], [26] use knapsack solvers or ILP (Integer Linear Programming) to address the issue of time constraint in regression testing. As the main aim of testing is to find out whether the delivered product meets customer's requirement, RBT (Requirement Based Testing) [13] is becoming popular. Model based prioritization mechanism [14], [33] is another possibility of addressing TCP problem where specification models (event state diagram/activity diagram) represent system behavior and the model execution information is used to order test cases. TCP techniques for GUI/Web based Applications has the edge of using user session data as test data [23].

All of the previous studies mainly used C programs or case study applications or web applications for implementing the prioritization prototypes. However, there are very few studies [4], [24], [25] which operated on testing object oriented programs. TCP techniques those are applied for C programs might not be fully applicable on object oriented programs because they do not take object oriented features into consideration. Panigrahi and Mall built a dependence model based prioritization prototype for object oriented programs [25]. We would like to bring up the work [4] conducted by Do *et al.* in 2006, which for the first time focused on prioritizing JUnit test cases. Even though JUnit is a very popular unit testing framework/paradigm for Java programs, a handful of studies [4], [26], [28], [29] applied Test Case Prioritization techniques for JUnit test cases. The study conducted by Do *et al.* [4] ran JUnit test cases on four java programs (*Ant*, *JMeter*, *Xml-security* and *Jtopas*). Zhang *et al.* designed Eclipse IDE plug-in JTop which statically reads a Java program under test [28]. In 2012, Mei *et al.* introduced JUPA (JUnit Test Case Prioritization Techniques operating in the Absence of coverage information) [29]. JUPA can be

applied in test class and test method level. It operates best at test method level and with feedback based strategies.

Even though TCP techniques without coverage information has shown promising results, it is still not that popular. Collecting coverage information is a costly effort in terms of time, effort and cost [37]. Because of this, many researchers are trying to avoid this step. Towards this effort, prioritization using optimization techniques (genetic algorithms, ant colony optimization) [35], [39] are gaining momentum. Several meta-heuristic and evolutionary search algorithms were explored by Li *et al.* in 2007. The results of their study indicated that genetic algorithms are highly effective in multimodal search space of regression testing [30]. Five search algorithms (Total Greedy, Additional Greedy, 2- Optimal Greedy, Hill Climbing and Genetic Algorithms) were explored by Li *et al.* in 2010 [31]. Conrad *et al.* built a new framework [32] called GELATIONS (GENeticALgorithmBased Test suiteprioritization System). Williams and Kapfhammer conducted a study to compare the effectiveness of search based and greedy prioritizers [34]. For large test suites, hill climbing algorithm appeared more efficient than greedy algorithm. The prospect of ant colony optimization technique was explored by Agrawal and Kaur for test case selection [35]. In 2015, Epitropakis *et al.* [6] studied multi objective prioritization mechanism which considered three major objectives – average percentage of coverage achieved, average percentage of coverage of changed code and average percentage of past fault coverage. Multi objective prioritization mechanism depicted remarkable improvement in APFD values. But their subject of study was limited to C/C++ programs only. In 2016, Marchetto *et al.* [27] designed another multi-objective prioritization mechanism which aims to maximize the number of detected faults as well as detect critical faults.

Considering all the previous studies, we surmised to research the rarely explored avenue of prioritizing JUnit test cases using optimization techniques with novel fitness objectives.

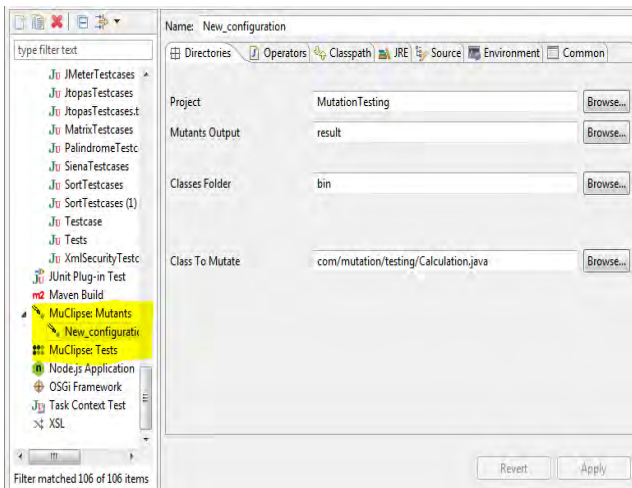
III. RESEARCH METHODOLOGY

A. SUBJECT PROGRAMS OF STUDY

To investigate the effectiveness of our approach, we chose a wide variety of Java programs. 10 Java objects with several mutated versions were studied. Table 1 enlists the programs with their size and statistics. Among these programs, the versions of *Ant* (build tool for Java programs), *Jtopas* (parses text data), *Xml-security* (maintains encryption security standards for XML) and *JMeter* (load testing tool) are downloaded from SIR (Software Artifact Infrastructure Repository). For each case, we chose the last released version. *NanoXML* (Java XML parser solution) was also retrieved from SIR. The rest of the objects are mainly obtained from SourceForge(<http://sourceforge.net>) and Apache Jakarta (<http://jakarta.apache.org>) websites. While *JDepend* generates design quality metrics for Java package, *JodaTime* acts as a substitute for Java date and time class. The *TimeMoney*

TABLE 1. Statistics of chosen subject programs.

Subject Program	KLoC	Number of methods	Executed test cases	Number of mutants
Ant-v8	80.5	7524	58	10
Jtopas-v3	5.4	748	56	12
Xml-security-v3	17	1398	55	15
JMeter-v5	41	3613	51	10
JDdepend	2.9	335	51	8
JodaTime	33	4025	55	6
Siena	59.84	1895	55	8
TimeMoney	2.68	505	52	10
CommonsLang	23.5	2156	56	8
Nanoxml	7.6	134	51	12

**FIGURE 1.** Configuring mutants using MuClipse.

project helps in creating code for recurring areas like time, money etc. *CommonsLang* (<https://commons.apache.org/>) provides methods for string manipulation, concurrency etc.

The mutants were generated using Eclipse plug-in MuClipse. Fig. 1 showcases a screenshot for mutant creation. We used several traditional operators (AOR - Arithmetic operator replacement, UOI - Unary operator insertion, ROR - Relational operator replacement) from MuClipse. We also utilized different object oriented operators (IOD - Overriding method deletion, OMD - Overloading method deletion) etc.

B. THREATS TO VALIDITY

- The considered subject programs, generated test cases and produced mutants are well sampled but may pose threats to validity. They vary widely in size and functionality, but they are homogeneous in nature. By this, we mean that, they represent the same object oriented language Java.
- Even though the generated mutants are prudently experimented, they might vary from programs with real faults. As large commercial/industrial programs [38], [40] behave differently, finding sufficient objects of study

including programs, faulty versions of programs, test case data is a challenge for this study.

- Our work also carries the limitations of addressing situations of ties or draws. If any two test cases contribute the same value, then the test case which will be assigned higher priority is randomly decided.
- Using the APFD metric to measure the effectiveness of the optimization algorithms might be a threat to construction validity. As APFD metric has its own limitations [1], additional studies are required to explore other prioritization metrics.

IV. EXPERIMENTAL DESIGN AND IMPLEMENTATION

This section describes the initial set up and implementation of each optimization technique in detail.

Step 1: In the very first step, faulty versions of each program (mutants) were prepared using *MuClipse*. On an average, we prepared 10 mutants for each program. For example, Mutant 2 of Ant program detected the fault of passing an argument which started with “-d”. If the user passed an argument as “-diagnostics”, then a diagnostic report was generated. Mutant 1 of JMeter program revealed that if non-numeric values had been passed to check the http response code, then a fault will be thrown. Mutant 6 of Xml-security revealed that if the input was null then a fault had occurred. Mutant 5 of JDdepend exposed the fault of null input zip file which had been sent for parsing. Mutant 2 of JTopas revealed that if the input data started with a special sequence or a null string value, then a fault will be thrown. Mutant 7 of JodaTime program showed that if a user wanted to parse a date but had sent a non-numeric input, then a fault had occurred.

Next, a set of JUnit test cases were generated for each program. JUnit test suite has many test classes which in turn have many test methods (Fig. 2). JUnit test case sequence is basically a chain of method calls. Reordering each test-method level test cases within a JUnit test class is more granular than reordering test-class level test cases. So, in this study we focused on prioritization of test method level test cases. Our study assumes the fact that the test methods are independent to each other and have no execution dependency. Following is a code snippet which resembles two JUnit test cases which were composed while testing JMeter program for load testing. While the first test case finds the maximum of given numbers, the second test case checks the cube of a given number. Test methods are annotated by @Test annotation.

@Test

```
public void testFindMax(){
    System.out.println("test case find max");
    assertEquals (-2,Logic.findMax(new int[]{-12,-3,
-4,-2}));
}
```

@Test

```
public void testCube(){
    System.out.println("test case find cube");
    assertEquals (23,Logic.cube(3));
}
```

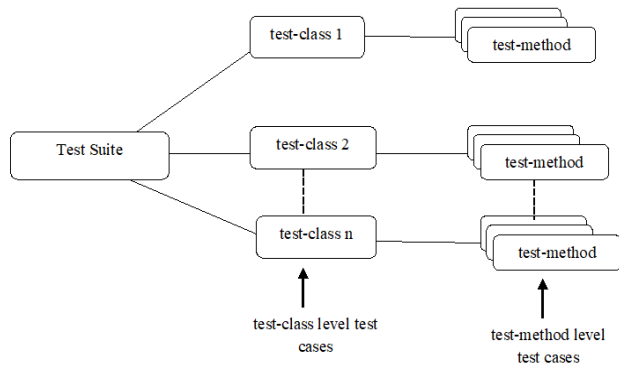



FIGURE 2. JUnit test suite structure.

Step 2: After preparing the mutants and test cases, we gathered the modification information between the base program version and the mutated program versions. The ‘diff’ utility of UNIX was utilized for this purpose. Collecting this modification information will help us in building our fitness metric which aims at maximizing number of modified lines executed per unit of time. If P is the original program and P' is the modified program, then the modifications between P and P' is stored in a file named log.txt. Apart from using *diff*, static monitoring was also done to exclude cases like variable renaming, method renaming etc. The changed portions of each program were inspected and the number of modified lines executed by each test case was recorded.

Step 3: Finally, we logged execution time of each JUnit test case. The execution time of each test case ranged in seconds or minutes unit. For some programs (*JDepend*) the test cases got executed in short execution time while for some other programs (*Siena*) it took considerable execution time because of I/O interactions, initialization etc. As regression testing is performed in time restraint environments, our fitness metric is dependent upon the number of modified lines covered in unit amount of execution time. Following is a code snippet which records the start time and stop time while running a test case and hence finds the time taken to execute the test case.

```
@Test
public void testCase7()
{long startTime = System.currentTimeMillis();
 assertEquals ("Success",
 Program.getResult(String.valueOf(8)));
 long stopTime = System.currentTimeMillis();
 long timeElapsed = (stopTime - startTime);
 if (timeElapsed > 0)
 {testCaseDetails.addProperty("timeRequired",
 timeElapsed);}
}
```

Step 4: Next, we built JSON arrays to store each test case details (number of modified lines, execution time, pass/fail status etc.). This information will be useful input parameter for the studied optimization algorithms. As the data types involved integer, string, boolean etc. the usage of JSON array became evident for storing multiple data types.

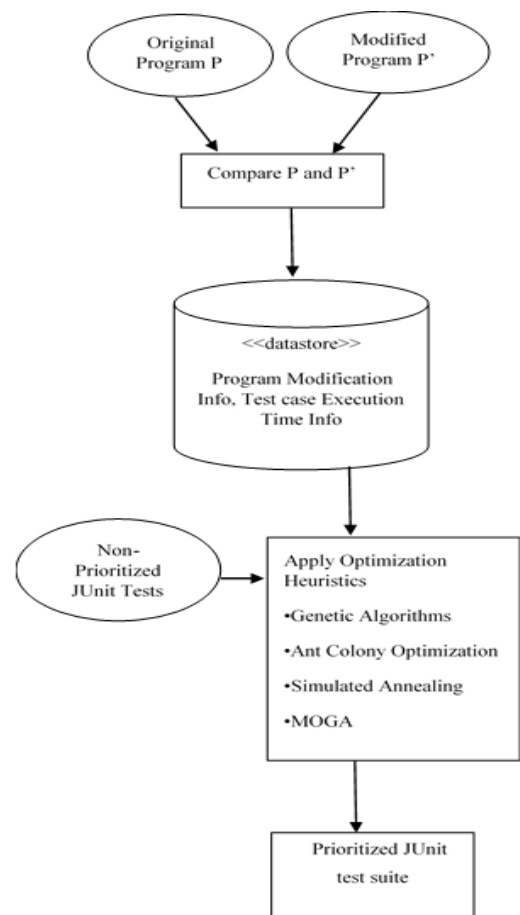


FIGURE 3. Overview of experiment process.

Fig.3 represent the basic block diagram of our experimental set up. In the subsequent subsections we will be detailing the application of several optimization heuristics for solving the test case prioritization problem.

A. APPLICATION OF GENETIC ALGORITHMS (GA)

The test case prioritization problem was replicated in context of 0/1 knapsack problem and then Genetic Algorithm (GA) was used to solve it. The 0/1 knapsack problem is an NP-complete problem. The formulation of the test case prioritization problem which is suitable to an optimization algorithm is a vital issue.

The 0/1 knapsack problem states that given a knapsack with fixed capacity and a collection of discrete items each with its own weight and value, the maximum cumulative value of items that fills the knapsack should be such that the sum of the weight of the items does not cross the knapsack's capacity. We mapped each JUnit test case with a knapsack item, the execution time of each test case as its weight, the number of modified lines executed by the JUnit test case as its value and the maximum amount of time allowed for running the prioritized test suite as the knapsack's maximum capacity.

After this mapping, we applied GA for prioritization. GA is a stochastic technique which emulates the process of natural

evolution and is based on the idea of selecting the fittest chromosome. In our case, the chain of test cases is the chromosome (For example, for 5 test cases t1, t2, t3, t4, t5 the non-prioritized sequence is t1- > t2- > t3- > t4- > t5) and our aim is to optimize this sequence. We first generated a random initial population size of 200. Then, each chromosome of the population was evaluated against a fitness value. The fitness/objective function is represented by a scalar quantity $f(x)$ where x is a real variable and it aims to maximize or minimize a certain goal.

We designed a novel fitness function as –

$$f1 = \sum_{i=1}^n \frac{O_i \times ML_i}{ET_i}$$

where, O_i is the order/position of the JUnit test case in the test case sequence generated by Genetic Algorithm, ML_i is the number of modified lines executed by the test case, ET_i is the execution time of the test case, and n is the number of JUnit test cases.

While O_i is the order generated by GA, the process of recording ML_i , ET_i is narrated in Section IV in several steps. For example, a test case sequence/chromosome is t2-> t3-> t4-> t1->t5 and t1, t2, t3, t4, t5 covers 16, 18, 13, 14, 12 number of modified lines respectively. Execution time of t1, t2, t3, t4, t5 are 3 minutes, 3 minutes, 2 minutes, 1 minutes and 4 minutes respectively (data taken based on Siena program). Then the fitness value for this sequence will be –

$$f1 = 4 \times \frac{16}{3} + 1 \times \frac{18}{3} + 2 \times \frac{13}{2} + 3 \times \frac{14}{1} + 5 \times \frac{12}{4} = 97.33$$

Our objective is to cover maximum number of modified lines per unit of execution time while running the regression test suite. This objective will promote *time aware test suite prioritization* where the test suite gets limited time to run. Next, by applying Roulette Wheel selector we selected chromosomes from the initial population on the basis of our designed fitness function. The test case sequence which produces higher fitness value is chosen over others. After selection, the partially matched crossover (PMX) technique was applied for recombination. The crossover probability (p_c) was chosen as 0.35. Finally, using swap mutator we found out the best phenotype/tuple of test cases with maximum fitness that gets executed within t_{max} as t_{max} is the maximum time limit allowed for running the prioritized test suite. In our case, we considered t_{max} as 50 minutes. This value was chosen empirically while running regression test suites comprising 47 test cases (on average) for each of our subject programs (ranging from 2.68 KLOC to 80.5 KLOC). The chosen mutation probability (p_m) was 0.10. As a high crossover rate may cause premature convergence (situation of local optima instead of global optimum) and a high mutation probability causes delayed convergence to any optimum solution, we chose low values of p_c and p_m . A fixed generation number (75 in our case) was used as the termination criteria.

Following algorithm summarizes the previous steps –

Algorithm 1 Genetic Algorithm to solve TCP problem

Input: Program P , Test suite T , Number of JUnit test cases (n) as knapsack item, Number of modified lines (ML) by a test case as knapsack item value, Execution time (ET) of each test case as knapsack weight, p_c as crossover probability, p_m as mutation probability, number of iterations as g_{it} , maximum time limit for execution of prioritized test suite t_{max} as knapsack's maximum capacity

Output: Best phenotype/maximum fitness tuple ($T_{prioritized}$) which gets executed within t_{max}

1: **begin**

2: Create initial population as IP

3: **for** each iterations in g_{it} **do**

4: Calculate fitness value as FV where

$$FV = \sum_{i=1}^n \frac{O_i \times ML_i}{ET_i}$$

5: Apply *SelectParentsMethod* (IP , FV) to generate selected parent population as SP

6: Apply *CrossoverMethod* (SP , p_c) to generate offspring population as OP

7: Apply *MutationMethod* (OP , p_m) to generate $T_{prioritized}$

8: **end for**

9: **return** $T_{prioritized}$

10: **end**

After obtaining new ordering of test cases from the maximum fitness tuple ($T_{prioritized}$), the effectiveness of obtaining ordering was represented in terms of APFD (Average Percentage of Faults Detected) metric described in Section V. APFD value ranges from 0 to 100 where higher value indicates faster fault detection ability. Table 6 to Table 15 enlists all the calculated APFD values for all the subject Java programs under consideration.

B. APPLICATION OF SIMULATED ANNEALING (SA)

The meta-heuristic simulated annealing was applied for test case prioritization. As basic hill climbing very often converges on a local maximum, we examined simulated annealing. While exploring the search space, the algorithm starts with an arbitrary solution and instead of examining all nodes/neighbors it selects the next move/neighbor randomly. At every iteration, the energy function/target function is quantified. If the selection of the neighbor causes an uplift in the target function $f(x)$, then a move is made to that neighbor. If the selection of the neighbor does not cause an uplift in the target function $f(x)$, then also a move is made to that neighbor, provided it stays within the acceptance probability. Thus, a probabilistic decision is reached. At each step, the probability of selecting a better solution is increased (tending towards 1) and the probability of selecting a worse solution is decreased (tending towards 0). For solving the test case

prioritization problem using this algorithm we performed the following steps-

1. A random test case sequence (T_{Random}) is designated as the current sequence.
2. For each iterations of the simulated annealing algorithm, the value of energy function or target function $f(x)$ is calculated.
3. Any new ordering of JUnit test cases (T_{New}) is generated as neighbor and the value of the target function $f(x)$ is recalculated.
4. If the target function increases, then this new ordering (T_{New}) is considered as the current sequence.
5. If the target function decreases, then also the new test case ordering (T_{New}) is considered as the current sequence provided it lies within the worse solution acceptance probability. Otherwise, the previous ordering is considered as the current sequence.
6. The algorithm stops at the end of allotted time budget for running prioritized test suite (50 minutes in this case).

For keeping uniformity and fair comparison, we kept the target function and allotted time budget as same as we designed for genetic algorithm implementation. Because of its ability to avoid local maxima convergence, solving the test case prioritization problem using simulated annealing algorithm appeared as a promising effort. Table 6 to Table 15 registers all the calculated APFD values obtained from the JUnit test case orderings generated by this technique for all the subject programs.

C. APPLICATION OF ANT COLONY OPTIMIZATION (ACO) TECHNIQUE

Our study also explored usage of ant colony optimization technique for solving time constraint test case prioritization problem for JUnit test suites. As an ant wanders for a food source, it deposits pheromone (a chemical substance) along the path it follows. While this pheromone trail helps other ants to reach the target food source, it also helps the ants to return back to their original starting point. If a new ant moves along the pheromone deposited path, then the concentration of the pheromone increases as the new ant puts more pheromone on the traversed path. However, the pheromone evaporates as time passes by. Thus, the path which accumulates maximum pheromone becomes the chosen path by other moving ant. The paths followed by ants build a connected graph $G(V, E)$ which is comprised of vertices V and edges E .

While applying the ACO technique for solving test case prioritization problem, we formulated the following mappings –

- As ants move from one node to another, each node (n)/vertex is a test case
- Maximum time allowed for an ant to reach the food/target is t_{max}
- Each JUnit test case executes certain number of modified lines (ML) in unit amount of execution time(ET)

TABLE 2. Sample test case data for siena program.

Test case	Number of Modified Lines (ML)	Execution Time(T) (in minutes)	ML/T
t1	15	3	5
t2	18	3	6
t3	16	4	4
t4	14	2	7
t5	10	5	2
t6	18	4	4.5
t7	15	3	5
t8	10	5	2
t9	12	4	3

TABLE 3. Test case execution in 1st iteration.

Test case sequence	Pheromone weight of each sequence
t2 t6 t3 t4	21.5
t7 t1 t2 t9	19
t7 t6 t5	11.5
t4 t3 t5	13
t4 t6 t7 t1	21.5

and this is equivalent to pheromone weight of each edge (p_w).

$$p_w = \sum_{i=1}^n \frac{ML_i}{ET_i}$$

So, the pheromone weight increases if more amount of modified lines get executed per unit of time.

- Pheromone deposition rate is +1 or 100% while each ant traverses the edge.
- Pheromone evaporation rate is $k\%$ of p_w (we considered $k = 15$)

Explanation With a Case Study: Suppose, for the subject program *Siena*, we develop a test suite T with 9 test cases. Let the initial execution ordering be $\{t1, t2, t3, t4, t5, t6, t7, t8, t9\}$. Table 2 enlists the execution time, number of modified lines covered by each test case, corresponding target function value etc. Maximum time allowed for prioritization of the test suite (t_{max}) is 50 minutes (kept it same as knapsack problem). This time constraint is also the stopping condition of ACO execution of *Siena* program. The various paths followed by ants during first iteration is presented in Table 3. Among the followed paths, path $\{t2 t6 t3 t4\}$ and path $\{t4 t6 t7 t1\}$ produces highest weight of pheromone. So, both of these paths are selected from all the traversed paths and this is depicted in Fig. 4.

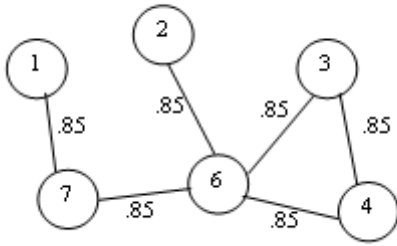


FIGURE 4. Execution path after 1st iteration.

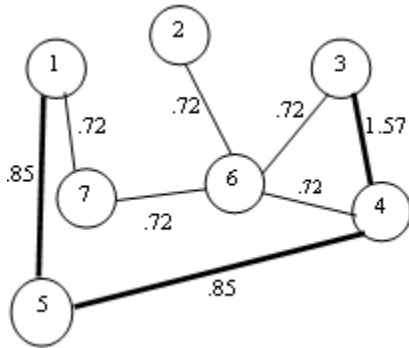


FIGURE 5. Execution path after 2nd iteration.

The pheromone deposition rate is + 1 and evaporation rate is 15%. So, after first traversal the edge weight becomes $(1 - 0.15) = 0.85$. In 2nd iteration, path {t3 t4 t5 t1} gets selected from all the traversed paths enlisted in Table 4 and this new path is added in Fig. 5. As edge 3-4 is also traversed in this iteration, its weight increases to $\{(.85 + 1) - (.15 \times .85)\} = 1.57$. Because of pheromone evaporation, the weight of already existing edges reduce to $\{0.85 - (.15 \times .85)\} = 0.72$. The path which has highest pheromone weight becomes the best path in each iteration. In 3rd iteration, path {t3 t4 t5} gets selected from all the traversed paths enlisted in Table 5 and this new path is added in Fig. 6. Because of addition of this edge, the pheromone weight increases in already existing edge 3-4 and edge 4-5. The ACO technique stops running after 3rd iteration because the time limit reaches 50 minutes. Thus the best path/test case ordering which gets executed within t_{max} (maximum time allowed for prioritization) and covers maximum number of modified lines per unit of time is discovered to be {t3 t4 t5}.

Using ACO technique, test cases can be effectively selected and prioritized. We calculated test case ordering for all the subject Java programs by the above mentioned ACO technique. APFD values were calculated from the orderings of test cases. Table 6 to Table 15 registers all the calculated APFD values for all the subject Java programs for this technique.

D. APPLICATION OF MOGA

The limitations of single objective regression test ordering has encouraged us to explore MOGA (Multi-objective GA). Other than the previous fitness metric (maximize the execution of number of modified lines per unit of execution time),

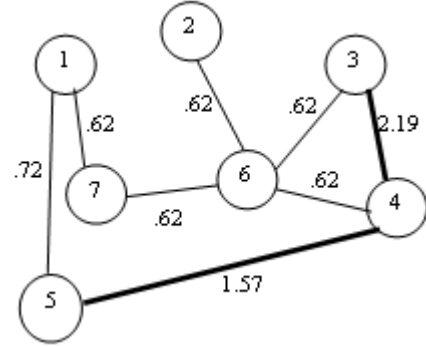


FIGURE 6. Execution path after 3rd iteration.

TABLE 4. Test case execution in 2nd iteration.

Test case sequence	Pheromone weight of each sequence
t8 t6 t3 t5 t1	17.5
t5 t3 t7 t1	16
t4 t3 t2	17
t3 t4 t5 t1	18

TABLE 5. Test case execution in 3rd iteration.

Test case sequence	Pheromone weight of each sequence
t8 t7 t9 t5	12
t3 t4 t5	13
t6 t5 t7	11.5
t3 t5 t2	12

we designed another target objective which aims to maximize number of inheritance edges executed by a test case. Test case prioritization for object oriented programs need to take object oriented features into account and our study intends to address this issue. A python script was used to calculate the number of inheritance edges executed by each test case. The program modification information and inheritance information were stored in matrix form in the MOGA implementation. Following is a code snippet which stores program modification data in modificationInfoMatrix[][] and inheritance data in inheritanceMatrix[][].

```
for (int i = 1; i < n; i++) {
    int x;
    int y;
    x = solution.getVariableValue(i);
    y = solution.getVariableValue(i + 1);
    fitness1 += modificationInfoMatrix[x][y];
    fitness2 += inheritanceMatrix[x][y];
}
```

The two objective functions for maximization can be summarized as -

$$f1 = \sum_{i=1}^n \frac{O_i \times ML_i}{ET_i} = x1$$

TABLE 6. APFD values (in %) for several mutants of ANT-V8.

Mutants for Ant-v8	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	82.3	78.2	98.7	90
Mutant2	87.6	78.9	96.5	89.1
Mutant3	89	76.5	97.4	92.1
Mutant4	83.1	73.2	93.2	93.2
Mutant5	87.2	79.1	95.1	94.5
Mutant6	84.3	72.1	97.9	96.2
Mutant7	85.3	83.5	98.3	87.1
Mutant8	87.9	85.5	99.3	86.4
Mutant9	85.2	84.4	97.1	93.2
Mutant10	88.2	81.9	99.2	91.9

TABLE 7. APFD values (in %) for several mutants of Jtopas-v3.

Mutants for Jtopas-v3	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	87.2	88.7	86.4	99.8
Mutant2	87.5	89.3	86.1	98.6
Mutant3	89	90.8	89.7	99
Mutant4	83.6	91.1	89.4	99.8
Mutant5	91.9	82.1	90.1	96.5
Mutant6	94.3	87.6	91.5	99.4
Mutant7	92.7	85.5	82.1	98.5
Mutant8	91.1	87.5	80.1	99.1
Mutant9	90.2	90.8	94.5	99.5
Mutant10	89.1	91.2	92.3	99.9
Mutant11	88.1	82.8	82.4	98.9
Mutant12	82.1	86.9	88.7	99.6

where, O_i is the order/position of the JUnit test case in the test case sequence, ML_i is the number of modified lines executed by the test case, ET_i is the execution time of the test case, n is the number of JUnit test cases.

$$f2 = N$$

where, N is the number of inheritance edges covered by the test case.

The widely recognized NSGA-II (Non-dominated Sorting Genetic Algorithm II) algorithm was run on all the subject programs with a population size of 200 for 75 generations. The pareto optimal region for this run lies in the range of $0 \leq x1 \leq 100$ and $0 \leq N \leq 10$. These values are chosen empirically based on the NSGA-II execution for all the subject programs. The algorithm converged to the optimal range with a population distribution. Fig. 9 represents a convex Pareto optimal front for the *Ant* program. Test cases which constitute the pareto-optimal front are shown by the dotted part of the graph.

V. RESULTS AND ANALYSIS

The effect of application of each optimization heuristics is represented in terms of APFD metric, APFD graph, box plot and ANOVA analysis.

TABLE 8. APFD values (in %) for several mutants of Xml-security-v3.

Mutants for Xml-security-v3	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	81.2	80	83.5	98.2
Mutant2	81.9	84.9	83.7	99
Mutant3	82.3	81.1	83.8	98.9
Mutant4	82.7	81.6	82.1	98
Mutant5	84.2	87.5	81.1	98.1
Mutant6	86.1	82.7	80.3	98.1
Mutant7	81.9	86.4	82.4	98.3
Mutant8	88	81.1	83.2	98
Mutant9	82.3	81.7	83.2	99.2
Mutant10	81.8	80.3	81.5	99.2
Mutant11	82.7	86.5	87.6	97.9
Mutant12	81.9	82.4	82.5	99
Mutant13	80	80.5	88.9	99.1
Mutant14	82.4	82.3	81.4	97.8
Mutant15	85	81.4	81.2	97.9

TABLE 9. APFD values (in %) for several mutants of JMeter-v5.

Mutants for JMeter-v5	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	90.2	87.6	98.7	92.1
Mutant2	99.8	85.4	92.1	92.3
Mutant3	90.1	75.6	94.2	93.3
Mutant4	76.5	71.1	92.1	94.6
Mutant5	89.7	69.2	93.4	97.5
Mutant6	90.8	70.2	94.2	91.7
Mutant7	90.3	76.4	98.9	90.9
Mutant8	91.2	71.9	95.9	89.9
Mutant9	90.1	70.2	89.8	91.7
Mutant10	89.7	77.2	93.5	92.7

TABLE 10. APFD values (in %) for several mutants of JodaTime.

Mutants for JodaTime	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	75.8	80.5	90.9	97.6
Mutant2	81.2	81.2	97.6	95.5
Mutant3	80	76.5	89.7	96.1
Mutant4	75.1	79.8	81.2	97.5
Mutant5	70.2	87.6	92.3	94.5
Mutant6	67.5	69.5	94.4	98.7

A. CALCULATION OF APFD METRIC

APFD (Average Percentage of Faults Detected) measures the weighted average of percentage of faults detected during the execution of the test suite. APFD value ranges from 0 to 100 where higher value indicates faster fault detection ability [1]. As each optimization techniques generated different ordering of test cases, APFD values were calculated for all the mutants

TABLE 11. APFD values (in %) for several mutants of JDepend.

Mutants for JDepend	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	81.3	91.2	87.6	90.2
Mutant2	83.2	91.7	86.5	89.8
Mutant3	88.7	90.8	89.1	87.6
Mutant4	81.2	89.7	90.8	85.4
Mutant5	81.6	92.5	88.7	92.3
Mutant6	80.9	94.3	88.2	95.3
Mutant7	79.2	98.1	81.2	93.1
Mutant8	81.2	88.2	89.7	87.3

TABLE 12. APFD values (in %) for several mutants of Siena.

Mutants for Siena	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	88.7	76.5	84.8	98.4
Mutant2	89.3	82.1	87.6	99.7
Mutant3	86.5	70.9	85.5	99.6
Mutant4	84.3	69.8	91.2	99
Mutant5	78.6	71.2	90.8	98.3
Mutant6	94.3	70.9	79.5	99.6
Mutant7	90.8	82.3	81.4	98.9
Mutant8	88.7	81.1	76.5	97.9

TABLE 13. APFD values (in %) for several mutants of TimeMoney.

Mutants for TimeMoney	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	97.6	79.4	83.4	94.7
Mutant2	94.5	79.6	84.4	96.5
Mutant3	95.7	79.9	85.6	98.7
Mutant4	93.4	80.5	87.6	94.6
Mutant5	96.7	78.6	86.5	93.2
Mutant6	95.9	84.6	90.3	92.5
Mutant7	97.4	85.6	89.4	91.9
Mutant8	99.4	77.3	88.5	97.8
Mutant9	95.3	82.5	83.2	98.7
Mutant10	96.7	80.5	81.3	96.7

TABLE 14. APFD values (in %) for several mutants of CommonsLang.

Mutants for CommonsLang	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	78.6	70.3	88.6	98.6
Mutant2	79.3	76.2	89.5	96.7
Mutant3	76.8	71.2	89.1	95.4
Mutant4	87.5	73.4	91.8	97.6
Mutant5	81.1	74.5	89.7	98.7
Mutant6	82.3	68.7	88.2	98.2
Mutant7	83.4	69.9	87.2	96.8
Mutant8	73.2	70.2	88.7	99.1

for all the 10 subject Java programs using the formula –

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{m \times n} + \frac{1}{2 \times n}$$

TABLE 15. APFD values (in %) for several mutants of Nanoxml.

Mutants for Nanoxml	APFD produced by GA	APFD produced by SA	APFD produced by ACO	APFD produced by MOGA
Mutant1	85.6	78.7	96.4	92.8
Mutant2	86.5	76	89.1	98.6
Mutant3	87.9	78.8	99.7	97.5
Mutant4	83.6	75.9	94.1	99.8
Mutant5	87.3	82.1	99.5	89.3
Mutant6	91.3	82.1	91.5	89.4
Mutant7	91	75.3	98.9	93.5
Mutant8	91.1	70.4	99.2	99.1
Mutant9	79	71.8	94.5	89.5
Mutant10	89.1	70.8	92.3	89.9
Mutant11	88.1	73.5	92.4	87.9
Mutant12	82.4	71.9	98.7	83.8

TABLE 16. Mean APFD Score (in %) for all subject programs for all techniques.

Program Name	APFD by Non-Prioritized Ordering	APFD by GA	APFD by SA	APFD by ACO	APFD by MOGA
Ant-v8	62.3	86.01	79.33	97.27	91.37
Jtopas-v3	67.9	88.9	87.85	87.775	99.05
Xml-security-v3	76.3	82.96	82.69	83.09	98.44
JMeter-v5	56.4	89.84	75.48	94.28	92.67
JDepend	75.4	82.16	92.06	87.72	90.12
JodaTime	77.9	74.96	79.18	91.01	96.65
Siena	63.5	87.65	75.6	84.66	98.92
TimeMoney	54.2	96.26	80.85	86.02	95.53
CommonsLang	52.1	80.275	71.8	89.1	97.63
Nanoxml	61.9	86.90	75.60	95.52	92.59

where, T is test suite under evaluation, m is number of faults detected, n is number of test cases, TF_i is the position of the first test in T that exposes fault i.

The single objective optimization heuristics (GA, ACO, Simulated Annealing) returned a single ordering of test cases (maximum fitness tuple) as the best phenotype. But the application of MOGA yielded a set of ordering that represent the pareto front. The average APFD value of all the solutions is taken as the final APFD value for comparison with other technique. Table 6 to Table 15 registers all the calculated APFD values from the generated test case orderings for several mutated versions of the programs under consideration. Table 16 acts as a summary table which summarizes the mean APFD of all the mutants for all the subject programs. This will help us to get a single shot snap of the derived results.

Observing the calculated APFD values, it can be inferred –

- While Non – prioritized test case ordering yielded an average APFD of 64.79 %, MOGA based prioritization yielded an average APFD of 95.29%, ACO based

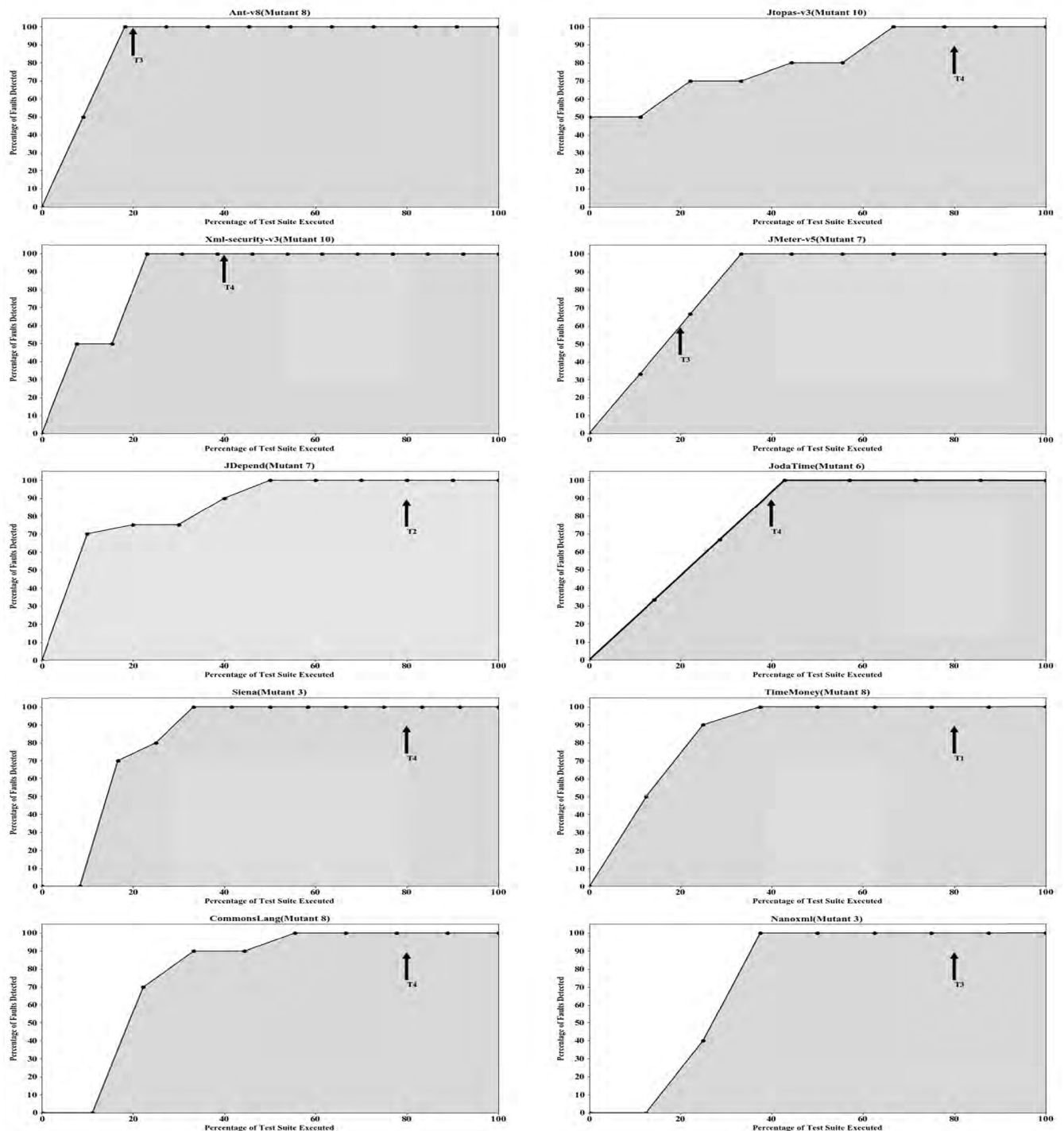


FIGURE 7. APFD Graph for several mutants of subject programs. In each case, the mutant which produced highest APFD value is chosen. For each mutant, the technique which produced highest APFD is plotted. X axis indicates percentage of test suite executed and Y axis indicates percentage of faults detected. T1: Prioritization by GA, T2: Prioritization by Simulated Annealing, T3: Prioritization by ACO, T4: Prioritization by MOGA.

prioritization produced an average APFD of 89.64 %. On the other hand, single objective GA based techniques generated an average APFD of 85.59 %. These values are generated by performing the mean of the APFD values summarized in Table 16. Based on these

results, it appears clearly that application of optimization heuristics produce great benefit for ordering JUnit test cases at test method level.

- MOGA appears to be the overall winner as 5 (*Jtopas*, *Xml-security*, *JodaTime*, *Siena*, *CommonsLang*) out of

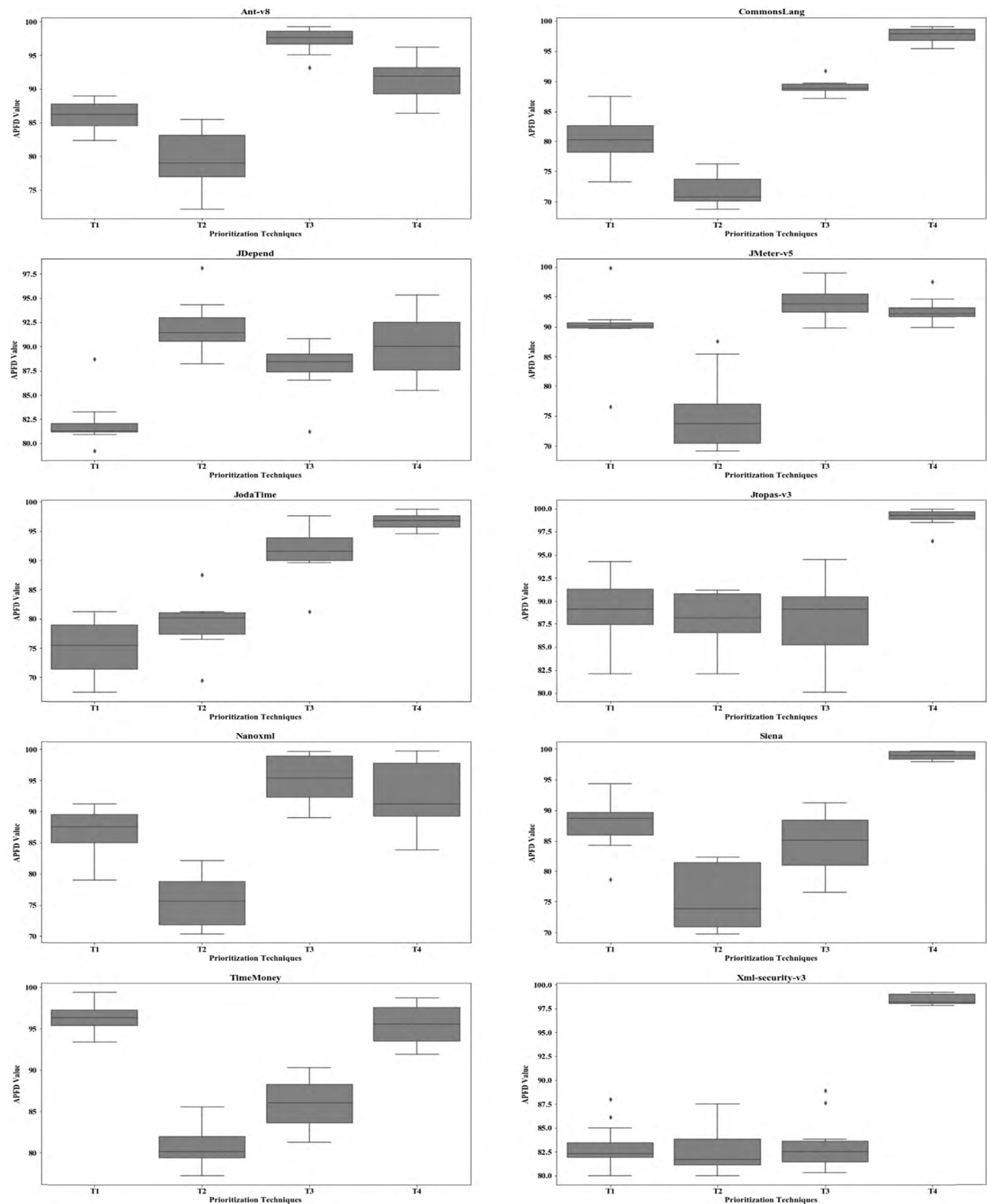


FIGURE 8. APFD Boxplots for each subject program. T1: Prioritization by GA, T2: Prioritization by simulated annealing, T3: Prioritization by ACO, T4: Prioritization by MOGA.

the 10 subject programs yielded highest mean APFD score using it.

- Among the single objective techniques, ACO outperformed others. It produced better result than GA and Simulated Annealing. 3 (*Ant*, *JMeter*, *Nanoxml*) out of the 10 subject programs showed best results with the application of ACO.
- For subject program *Ant*, prioritization using ACO generated highest average APFD value of 97.27% (from Table 16). This is a significant improvement over previously existing technique of using coverage information. Results of [4] indicated highest APFD value of 87% for the *Ant* program using block-additional coverage based prioritization technique.
- Study [4] indicated highest APFD of 77% for the *JMeter* subject program using method additional coverage based prioritization technique. We obtained highest APFD of 94.28% for this program using ACO.
- However, the performance gain for *Jtopas* and *Xml-security* showed little improvement. Method total coverage based prioritization technique [4] produced highest APFD of 97% for *Xml-security* program. While we obtained highest average APFD of 98.44% using MOGA for this program. Block additional coverage based prioritization technique [4] produced highest APFD of 95% for *Jtopas* program. We obtained highest average APFD of 99.05% using MOGA for *Jtopas*. So, in these two cases the improvement is marginal.
- In another study conducted by Alsbaugh *et al.* [26], the subject program *JDepend* produced highest APFD of 63.02% using method level coverage based prioritization. We obtained highest APFD of 92.06% for *JDepend* program using Simulated Annealing. This is a notable gain over the previously applied process.
- For rest of the subject programs (*JodaTime*, *Siena*, *TimeMoney*, *CommonsLang*, *Nanoxml*), we didn't find any study which explored application of GA, Simulated Annealing, ACO or MOGA for prioritizing JUnit test cases. According to our study, the results obtained by prioritization using optimization heuristics for all these subject programs appeared very promising. While MOGA appeared as the winner for *JodaTime*, *Siena*, *CommonsLang* program with highest average APFD value of 96.65%, 98.92% and 97.63% respectively, GA outperformed others in case of *TimeMoney* program. On the other hand, the subject program *Nanoxml* showed best result with ACO.

Re-execution of all scheduled test cases is impossible because of constraint testing time window. We hope the positive results of our study will make usage of optimization algorithms as a promising option for prioritizing JUnit test cases at test method level.

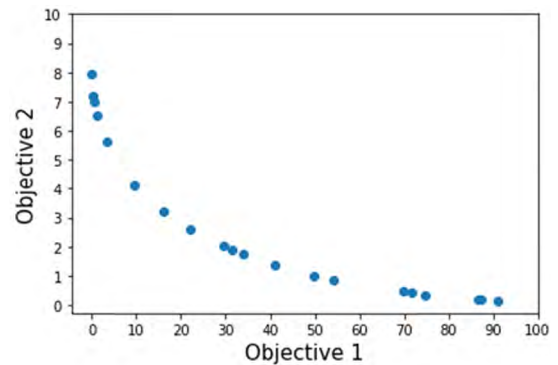


FIGURE 9. Pareto Front for NSGA II run of *Ant* subject program at iteration 50.

B. STATISTICAL ANALYSIS AND APFD GRAPH

APFD graphs are plotted in Fig.7 for all the 10 benchmark Java programs. Plotting APFD is basically showing the percentage of faults detected versus the percentage of test suite executed. In graphs, the X axis indicates percentage of test suite executed and Y axis shows percentage of faults detected. The curvature indicates cumulative percentage of faults detected. By this, we mean that if there are 4 faults then each will weight 25 as $(100/4)$. If the first test case detects two faults, then it becomes $25 * 2 = 50$ in Y axis. If the third test case detects any new fault other than those two previously captured faults, then the weightage becomes $50 + 25 = 75$. However, if the third test case doesn't detect any new fault, then the weightage stays at 50 only.

Data distribution in box plot charting (Fig. 8) also indicated the effectiveness of the application of optimization heuristics. Box plots are efficient way of representing data distribution where the box height spans the mean of the data. The vertical lines elongated from the boxes are called whiskers. The upper and the lower whiskers indicate one-fourth and three-fourth quartile range of data points.

We also performed one way ANOVA F-test analysis.

H_0 (Null hypothesis): APFD value for all techniques are same.

H_1 (Alternate hypothesis): APFD value for all techniques are different.

ANOVA F-test indicated that the techniques are significantly different considering the level of significance as 0.05. As rejection of null hypothesis shows that all techniques yield statistically diverse APFD values, the box plot visualizes the variation quantitatively.

The pareto front for the *Ant* program for NSGA II run is depicted in Fig.9. The non-dominated test cases constitute the pareto front. While increasing the APFD value is the ultimate goal, maximizing the two fitness objectives is the intermediate goal. Many researchers [30] have mentioned this intermediate goals as surrogate measures.

VI. CONCLUSION AND FUTURE SCOPE

As Test Driven Development (TDD) is generating great profits, testing is not considered as an overhead activity anymore. With the advent of *agile regression testing*, where test cases get executed after each sprint/delivery window, it becomes a mandate to validate all possible modified lines in time constraint environments. Thus, our choice of fitness metric which aims to maximize the execution of number of modified lines per unit of time is highly prudent in context of agile regression testing. If conducted offline, Test Case Prioritization will save time and cost and will not become an overhead. As we took another fitness objective of maximizing number of inheritance edges executed by a test case, our approach overcomes the limitations of single-objective prioritization techniques and leverages object oriented features. While collecting coverage information is an arduous activity, we hope the approach of ordering JUnit test cases using optimization heuristics will gain momentum. Also, the promising results of our study in comparison with the results obtained by previous studies [4] (where JUnit test cases were ordered using coverage information) made us confident towards this effort. While MOGA based prioritization yielded an average APFD of 95.29%, ACO based prioritization produced an average APFD of 89.64 %. On the other hand, single objective GA based techniques generated an average APFD of 85.59 %. The chosen Java programs, generated test cases and produced mutants are well sampled and support the positive outcome of the calculated APFD values. As we examined a good number of optimization heuristics (Single-objective GA, Simulated annealing, ACO, Multi-objective GA) for ordering JUnit test cases, the diversity of the considered optimization techniques ensured the validity of the trend.

In future, following directions should be investigated –

- We want to apply our strategy for large real time systems. Real regression faults are tough to locate and might behave differently. Regression testing of real time system is heavily time constrained as their simulation environment is very demanding and hosts multiple projects. We would like to explore the application of optimization heuristics for real time systems.
- Our study also includes the limitation of considering only one object oriented language (Java). We would like to overcome this by studying programs of other object oriented languages (C++, C#, PHP) in future.
- The study also has the limitation of exploring only one testing framework (JUnit). Other testing frameworks (TSL-Test Specification Language etc.) should be explored to judge the implementation potential of optimization heuristics.
- Also some other Multi Objective Evolutionary Algorithms (MOEA) approach like TAEA (Two Archive Evolutionary Algorithm) should be applied on JUnit test cases to do a comparative analysis with NSGA-II. As TAEA was specifically designed to overcome the limitations of NSGA-II, we would like to search whether it would show significant improvement in the results.

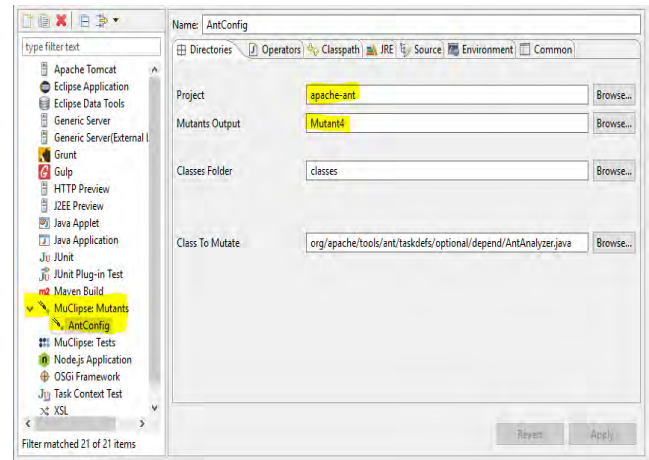


FIGURE 10. Mutant generation of *Ant* subject program using MuClipse.

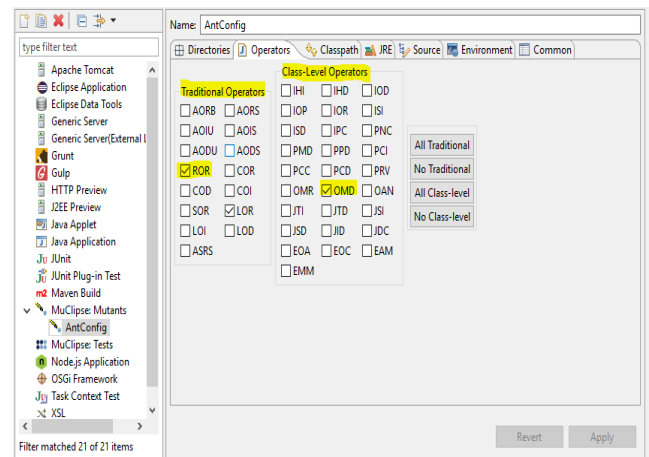


FIGURE 11. Usage of mutation operators (ROR, OMD) for mutant generation of *Ant* subject program using MuClipse.

We can confide this is the first thorough attempt to prioritize JUnit test cases using several optimization heuristics with the newly designed fitness objectives. We hope this article will be beneficial for professionals who have far-reaching interest in Test Case Prioritization domain with a focus on object oriented testing.

APPENDIX

We have highlighted some steps in this part to showcase the implementation details. Fig. 10 specifies the screenshot of Mutant 4 generation of the *Ant* subject program. Fig. 11 represents the usage of mutation operators ROR (Relational operator replacement) and OMD (Overloading method deletion) while generating this mutant.

After generating the JUnit test cases and the mutants, it was necessary to record all the test case details. This detail information of each test case will be used as an input by the leveraged optimization heuristics. JSON arrays were used to store each test case details (number of modified lines, execution time, pass/fail status etc.). Following is a code

snippet which showed the way of storing test case details for test case 5 of *Ant* subject program.

@Test

```
public void testCase5()
{
    long numOfLines = 40;
    long startTime =
        TimeUnit.NANOSECONDS.toMicros(System.nanoTime());
    String[] args = {"-help"};
    assertEquals("Success", MainFault.start(args, null, null));
    long stopTime =
        TimeUnit.NANOSECONDS.toMicros(System.nanoTime());

    JsonObject testCaseDetails = new JsonObject();
    JsonObject testCase = new JsonObject();
    testCaseDetails.addProperty("noOfLinesModified",
        numOfLines);
    long timeElapsed = (stopTime - startTime);
    if (timeElapsed > 0)
    {
        testCaseDetails.addProperty("timeRequired",
            timeElapsed);
    }
    else {testCaseDetails.addProperty("timeRequired", 1);
    }
    testCase.add("testCase5", testCaseDetails);
    jsonArr.add(testCase);
}
```

To the best of our knowledge, this is the first exhaustive attempt to prioritize JUnit test cases using several optimization heuristics with the newly designed fitness objectives. We hope these implementation details will be beneficial for professionals who have special interest in object oriented testing domain.

REFERENCES

- [1] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Trans. Softw. Eng.*, vol. 27, no. 10, pp. 929–948, Oct. 2001. doi: [10.1109/32.962562](https://doi.org/10.1109/32.962562).
- [2] J. A. Jones and M. J. Harrold, "Test-suite reduction and prioritization for modified condition/decision coverage," *IEEE Trans. Softw. Eng.*, vol. 29, no. 3, pp. 92–103, Mar. 2003. doi: [10.1109/TSE.2003.1183927](https://doi.org/10.1109/TSE.2003.1183927).
- [3] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Test case prioritization: A family of empirical studies," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 159–182, Feb. 2002. doi: [10.1109/32.988497](https://doi.org/10.1109/32.988497).
- [4] H. Do, G. Rothermel, and A. Kinneer, "Prioritizing JUnit test cases: An empirical assessment and cost-benefits analysis," *Empirical Softw. Eng.*, vol. 11, no. 1, pp. 33–70, 2006. doi: [10.1007/s10664-006-5965-8](https://doi.org/10.1007/s10664-006-5965-8).
- [5] H. Do and G. Rothermel, "On the use of mutation faults in empirical assessments of test case prioritization techniques," *IEEE Trans. Softw. Eng.*, vol. 32, no. 9, pp. 733–752, Sep. 2006. doi: [10.1109/TSE.2006.92](https://doi.org/10.1109/TSE.2006.92).
- [6] M. G. Ekitipakis, S. Yoo, M. Harman, and E. K. Burke, "Empirical evaluation of Pareto efficient multi-objective regression test case prioritization," in *Proc. Int. Symp. Softw. Test. Anal. (ISSTA)*, 2015, pp. 234–245. doi: [10.1145/2771783.2771788](https://doi.org/10.1145/2771783.2771788).
- [7] D. Hao, L. Zhang, L. Zang, Y. Wang, X. Wu, and T. Xie, "To be optimal or not in test-case prioritization," *IEEE Trans. Softw. Eng.*, vol. 42, no. 5, pp. 490–504, May 2016. doi: [10.1109/TSE.2015.2496939](https://doi.org/10.1109/TSE.2015.2496939).
- [8] J.-M. Kim and A. Porter, "A history-based test prioritization technique for regression testing in resource constrained environments," in *Proc. 24th Int. Conf. Softw. Eng. (ICSE)*, 2002, pp. 119–129. doi: [10.1145/581339.581357](https://doi.org/10.1145/581339.581357).
- [9] H. Park, H. Ryu, and J. Baik, "Historical value-based approach for cost-cognizant test case prioritization to improve the effectiveness of regression testing," in *Proc. IEEE 2nd Int. Conf. Secure Syst. Integr. Rel. Improvement (SSIRI)*, Jul. 2008, pp. 39–46. doi: [10.1109/SSIRI.2008.52](https://doi.org/10.1109/SSIRI.2008.52).
- [10] Y. Fazlalizadeh, A. Khalilian, M. A. Azgomi, and S. Parsa, "Prioritizing test cases for resource constraint environments using historical test case performance data," in *Proc. IEEE 2nd Int. Conf. Comput. Sci. Inf. Technol. (ICCSIT)*, 2009, pp. 190–195. doi: [10.1109/ICCSIT.2009.5234968](https://doi.org/10.1109/ICCSIT.2009.5234968).
- [11] A. G. Malishevsky, G. Rothermel, and S. Elbaum, "Modeling the cost-benefits tradeoffs for regression testing techniques," in *Proc. Int. Conf. Softw. Maintenance*, Oct. 2002, pp. 204–213. doi: [10.1109/ICSM.2002.1167767](https://doi.org/10.1109/ICSM.2002.1167767).
- [12] A. Srivastava and J. Thiagarajan, "Effectively prioritizing tests in development environment," *ACM SIGSOFT Softw. Eng. Notes*, vol. 27, no. 4, p. 97, 2002. doi: [10.1145/566171.566187](https://doi.org/10.1145/566171.566187).
- [13] H. Srikanth, L. Williams, and J. Osborne, "System test case prioritization of new and regression test cases," in *Proc. Int. Symp. Empirical Softw. Eng. (ISESE)*, Nov. 2005, pp. 64–73. doi: [10.1109/ISESE.2005.1541815](https://doi.org/10.1109/ISESE.2005.1541815).
- [14] B. Korel, L. H. Tahat, and M. Harman, "Test prioritization using system models," in *Proc. IEEE 21st Int. Conf. Softw. Maintenance (ICSM)*, Sep. 2005, pp. 559–568. doi: [10.1109/ICSM.2005.87](https://doi.org/10.1109/ICSM.2005.87).
- [15] D. Staff and M. D. Ernst, "Reducing wasted development time via continuous testing," in *Proc. 14th Int. Symp. Softw. Rel. Eng.*, Nov. 2003, pp. 281–292. doi: [10.1109/ISSRE.2003.1251050](https://doi.org/10.1109/ISSRE.2003.1251050).
- [16] B. Beizer, *Software Testing Techniques*, 2nd ed. New York, NY, USA: Van Nostrand, 1990.
- [17] H. Erdogmus, M. Morisio, and M. Torchiano, "On the effectiveness of the test-first approach to programming," *IEEE Trans. Softw. Eng.*, vol. 31, no. 3, pp. 226–237, Mar. 2005. doi: [10.1109/TSE.2005.37](https://doi.org/10.1109/TSE.2005.37).
- [18] G. Rothermel and M. J. Harrold, "Analyzing regression test selection techniques," *IEEE Trans. Softw. Eng.*, vol. 22, no. 8, pp. 529–551, Aug. 1996. doi: [10.1109/32.536955](https://doi.org/10.1109/32.536955).
- [19] W. E. Wong, J. R. Horgan, S. London, and A. P. Mathur, "Effect of test set minimization on fault detection effectiveness," *Softw., Pract. Exper.*, vol. 28, no. 4, pp. 347–369, Apr. 1998. doi: [10.1002/\(SICI\)1097-024X\(19980410\)28:4<347::AID-SPE145>3.0.CO;2-L](https://doi.org/10.1002/(SICI)1097-024X(19980410)28:4<347::AID-SPE145>3.0.CO;2-L).
- [20] G. Rothermel, M. J. Harrold, J. Ostrin, and C. Hong, "An empirical study of the effects of minimization on the fault detection capabilities of test suites," in *Proc. IEEE ICSM*, Bethesda, MD, USA, Nov. 1998, pp. 34–43. doi: [10.1109/ICSM.1998.738487](https://doi.org/10.1109/ICSM.1998.738487).
- [21] D. Hao, L. Zhang, and H. Mei, "Test-case prioritization: achievements and challenges," *Frontiers Comput. Sci.*, vol. 10, no. 5, pp. 769–777, 2016. doi: [10.1007/s11704-016-6112-3](https://doi.org/10.1007/s11704-016-6112-3).
- [22] L. Zhang, S.-S. Hou, C. Guo, T. Xie, and H. Mei, "Time-aware test-case prioritization using integer linear programming," in *Proc. 18th Int. Symp. Softw. Test. Anal. (ISSTA)*, 2009, pp. 401–419. doi: [10.1145/1572272.1572297](https://doi.org/10.1145/1572272.1572297).
- [23] A. M. Memon and Q. Xie, "Studying the fault-detection effectiveness of GUI test cases for rapidly evolving software," *IEEE Trans. Softw. Eng.*, vol. 31, no. 10, pp. 884–896, Oct. 2005. doi: [10.1145/1572272.1572297](https://doi.org/10.1145/1572272.1572297).
- [24] S. Panda, D. Munjal, and D. P. Mohapatra, "A slice-based change impact analysis for regression test case prioritization of object-oriented programs," *Adv. Softw. Eng.*, vol. 2016, May 2016, Art. no. 1. doi: [10.1155/2016/7132404](https://doi.org/10.1155/2016/7132404).
- [25] C. R. Panigrahi and R. Mall, "A heuristic-based regression test case prioritization approach for object-oriented programs," *Innov. Syst. Softw. Eng.*, vol. 10, pp. 155–163, 2014. doi: [10.1007/s11334-013-0221-z](https://doi.org/10.1007/s11334-013-0221-z).
- [26] S. Alspaugh, K. R. Walcott, M. Belanich, G. M. Kapfhammer, and M. L. Soffa, "Efficient time-aware prioritization with knapsack solvers," in *Proc. 1st ACM Int. Workshop Empirical Assess. Softw. Eng. Lang. Technol. (WEASEL Tech)*, IEEE/ACM 22nd Int. Conf. Autom. Softw. Eng. (ASE), 2007, pp. 13–18.
- [27] A. Marchetto, M. M. Islam, W. Asghar, A. Susi, and G. Scanniello, "A multi-objective technique to prioritize test cases," *IEEE Trans. Softw. Eng.*, vol. 42, no. 10, pp. 918–940, Oct. 2016.
- [28] L. Zhang, J. Zhou, D. Hao, L. Zhang, and H. Mei, "Jtop: Managing JUnit test cases in absence of coverage information," in *Proc. IEEE/ACM 24th Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2009, pp. 677–679.
- [29] H. Mei, D. Hao, L. Zhang, L. Zhang, J. Zhou, and G. Rothermel, "A static approach to prioritizing JUnit test cases," *IEEE Trans. Softw. Eng.*, vol. 38, no. 6, pp. 1258–1275, Nov./Dec. 2012.
- [30] Z. Li, M. Harman, and R. M. Hierons, "Search algorithms for regression test case prioritization," *IEEE Trans. Softw. Eng.*, vol. 33, no. 4, pp. 225–237, Apr. 2007.

- [31] S. Li, N. Bian, Z. Chen, D. You, and Y. He, "A simulation study on some search algorithms for regression test case prioritization," in *Proc. 10th Int. Conf. Quality Softw.*, Jul. 2010, pp. 72–81.
- [32] A. P. Conrad, R. S. Roos, and G. M. Kapfhammer, "Empirically studying the role of selection operators during search-based test suite prioritization," in *Proc. 12th Annu. Conf. Genetic Evol. Comput. (GECCO)*, 2010, pp. 1373–1380.
- [33] C. R. Panigrahi and R. Mall, "Model-based regression test case prioritization," *ACM SIGSOFT Softw. Eng. Notes*, vol. 35, no. 6, pp. 1–7, 2010.
- [34] Z. D. Williams and G. M. Kapfhammer, "Using synthetic test suites to empirically compare search-based and greedy prioritizers," in *Proc. 12th Annu. Conf. Companion Genetic Evol. Comput. (GECCO)*, 2010, pp. 2119–2120.
- [35] A. Agrawal and A. Kaur, "A comprehensive comparison of ant colony and hybrid particle swarm optimization algorithms through test case selection," in *Data Engineering and Intelligent Computing* (Advances in Intelligent Systems and Computing), vol. 542. Singapore: Springer, 2018, pp. 397–405.
- [36] R. K. Saha, L. Zhang, S. Khurshid, and D. E. Perry, "An information retrieval approach for regression test prioritization based on program changes," in *Proc. IEEE/ACM 37th Int. Conf. Softw. Eng.*, vol. 1, May 2015, pp. 268–279.
- [37] C. Henard, M. Papadakis, M. Harman, Y. Jia, and Y. Le Traon, "Comparing white-box and black-box test prioritization," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng.*, May 2016, pp. 523–534.
- [38] J. A. Parejo, A. B. Sánchez, S. Segura, A. Ruiz-Cortés, R. E. Lopez-Herrejon, and A. Egyed, "Multi-objective test case prioritization in highly configurable systems: A case study," *J. Syst. Softw.*, vol. 122, pp. 287–310, Dec. 2016.
- [39] S. N. Sivanandam and S. N. Deepa, *Principles of Soft Computing*. Hoboken, NJ, USA: Wiley, 2007.
- [40] D. Nardo, N. Alshahwan, L. Briand, and Y. Labiche, "Coverage-based regression test case selection, minimization and prioritization: A case study on an industrial system," *Softw. Test., Verification Rel.*, vol. 25, no. 4, pp. 371–396, 2015.



R. MUKHERJEE received the B.Tech. degree from the St. Thomas College of Engineering and Technology, India, in 2005, and the M.S. degree from the University of North Carolina, Charlotte, in 2009. She is currently pursuing the Ph.D. degree in software engineering. She has also worked in software industry for 6 years with prominent MNCs like IBM and Fuzzy Logix. She is a Research Scholar with Computer Science Department, BIT, Mesra, India. Her research interests include software testing, process modeling and automation, and predictive analytics domain. She received the National Merit Scholarship in 2001. During her Master's, she received Teaching Assistantship from UNCC. She was a member of Phi Kappa Phi Honor Society in 2009 for achievement in graduating class.



K. S. PATNAIK received the bachelor's degree in electrical engineering from NIT, Silchar, India, and the Master of Engineering degree in software engineering and the Ph.D. degree from BIT Mesra, Ranchi, India, respectively, where he is an Associate Professor with the Computer Science Department, with a research experience of 16 years and teaching experience of 14 years. almost 20 reputed journal publications and 21 conference publications. His research interests include software engineering, soft computing, machine learning, and the Internet-of-Things. He has conducted a national level symposium on "Computer Education: Needs and Expectations" in 2010 as an Organizing Secretary. He has also organized several national workshops on testing of object oriented software in 2011, high performance computing in 2016, and recent trends in software testing in 2019, which are sponsored by MCIT-DIT, GOI, LDRA, ACMISOFIT, OUP, and TEQIP, respectively. He is also a member of ISTE, IEL, CILab, University of Manitoba, Canada.

...